

OBSERVACIONES DE LA PRÁCTICA

Juan Sebastián Rojas Marín, 202610140, js.rojasm123@uniandes.edu.co

Katy Teran Mercado, 202614247, k.teranm@uniandes.edu.co

Juan José Ricon Santos, 202615789, jj.rincons1@uniandes.edu.co

Máquina 1	
Procesador	Intel(R) Core(TM) Ultra 5 235U (2.00 GHz)
Memoria RAM (GB)	16
Sistema Operativo	Windows 11

Tabla 1. Especificaciones de la máquina para ejecutar las pruebas de rendimiento.

Resultados

Carga de Catálogo LINEAR PROBING

Factor de Carga (PROBING)	Consumo de Datos [kB]	Tiempo de Ejecución Real @LP [ms]
0.1	NO COMPLETADO	82238,67
0.5	2607968,61	12697,28
0.7	1968559,07	13451,95
0.9	1616115,13	7852,33

Tabla 2. Comparación de consumo de datos y tiempo de ejecución para carga de catálogo con el índice por categorías utilizando LINEAR PROBING

Carga de Catálogo SEPARATE CHAINING

Factor de Carga (CHAINING)	Consumo de Datos [kB]	Tiempo de Ejecución Real @SC [ms]
2.00	1105585,51	7638,22
4.00	738400,08	6428,06
6.00	615629,80	6614,96
8.00	557264,82	6712,89

Tabla 3. Comparación de consumo de datos y tiempo de ejecución para carga de catálogo con el índice por categorías utilizando SEPARATE CHAINING

Gráficas

La gráfica generada por los resultados de las pruebas de rendimiento.

- Comparación de memoria y tiempo de ejecución para LINEAR PROBING y SEPARATE CHAINING

A partir de los resultados obtenidos, observamos una diferencia clara entre los dos mecanismos de manejo de colisiones. En linear Probing, tanto el tiempo como el consumo de memoria tuvieron cambios más notorios dependiendo del factor de carga utilizado. El caso más evidente fue el factor 0,1, cuyo tiempo promedio llegó a 82.238,67 ms, muy por encima de las demás pruebas.

Por otro lado, separate Chaining mostró tiempos mucho más estables, que se mantuvieron aproximadamente entre 6.400 y 7.600 ms. También presentó un menor consumo de memoria frente a los valores que obtuvimos con linear Probing. Esto nos permitió observar que, para la

carga de este catálogo y con nuestra implementación, separate Chaining tuvo un comportamiento más estable en las diferentes pruebas realizadas.

- Comparación de factor de carga y memoria para LINEAR PROBING y SEPARATE CHAINING

Al comparar el factor de carga con la memoria utilizada, encontramos una tendencia similar en ambos mecanismos: al aumentar el factor de carga máximo, el consumo de memoria disminuyó.

En linear Probing, la memoria pasó de 2.607.968,61 kB con un factor de 0,5 a 1.616.115,13 kB con un factor de 0,9. No incluimos el factor 0,1 en esta comparación porque su medición de memoria no pudo completarse.

En separate Chaining también observamos una disminución progresiva, pasando de 1.105.585,51 kB con factor 2,0 a 557.264,82 kB con factor 8,0. Por lo tanto, nuestras pruebas muestran que permitir un mayor nivel de ocupación de las tablas redujo la cantidad de memoria requerida para almacenar los datos.

Preguntas de análisis

1. ¿Por qué en la función **getTime()** se utiliza **time.perf_counter()** en vez de otras funciones como **time.process_time()**?

Utilizamos `time.perf_counter()` porque necesitamos conocer el tiempo real transcurrido durante la ejecución de las operaciones. Esta función ofrece una medición de alta precisión y tiene en cuenta todo el intervalo entre el inicio y el final de la operación. En cambio, `time.process_time()` mide principalmente el tiempo durante el cual el procesador estuvo trabajando sobre nuestro proceso. Por esta razón, `perf_counter()` resulta más apropiado para las pruebas de rendimiento que realizamos sobre la carga del catálogo.

2. ¿Por qué son importantes las funciones **start()** y **stop()** de la librería **tracemalloc**?

Estas funciones son importantes porque nos permiten controlar el periodo durante el cual queremos realizar el seguimiento de memoria. Con `tracemalloc.start()` comenzamos a registrar las asignaciones de memoria realizadas durante la ejecución, lo que posteriormente nos permite tomar las mediciones necesarias. Con `tracemalloc.stop()` detenemos ese seguimiento cuando ya terminamos la prueba. De esta manera podemos concentrar la medición en la operación que realmente queremos analizar y evitar mantener el seguimiento de memoria activo cuando ya no es necesario.

3. ¿Por qué no se puede medir paralelamente el **uso de memoria** y el **tiempo de ejecución** de las operaciones?

Durante nuestras pruebas pudimos notar directamente que activar `tracemalloc` hacía que la carga tardara considerablemente más. Esto ocurre porque el seguimiento de memoria también requiere trabajo adicional durante la ejecución, ya que Python debe registrar las asignaciones que se van realizando.

Por esta razón, si midiéramos memoria y tiempo al mismo tiempo, el tiempo obtenido no representaría únicamente el costo de cargar los datos, sino que también incluiría la sobrecarga producida por `tracemalloc`. Para evitar esta alteración, realizamos las mediciones de tiempo y memoria por separado.

4. Dado el número de elementos de los archivos (`large`), ¿Cuál sería el factor de carga para estos índices según su mecanismo de colisión?

En nuestro caso, los factores que analizamos dependen del mecanismo utilizado. Para linear Probing trabajamos con factores de 0,1; 0,5; 0,7 y 0,9. Estos valores son menores que 1 porque los elementos se almacenan directamente en las posiciones de la tabla y es necesario conservar posiciones disponibles para poder resolver las colisiones mediante sondeo.

Para separate Chaining trabajamos con factores de 2,0; 4,0; 6,0 y 8,0. En este caso podemos superar un factor de 1 porque una misma posición puede contener varios elementos dentro de su estructura asociada. Por esto, el factor de carga que puede manejar cada implementación está relacionado directamente con la forma en que resuelve las colisiones.

5. ¿Qué cambios percibe en el tiempo de ejecución al modificar el factor de carga máximo?

En linear probing encontramos cambios bastante marcados. Con un factor de 0,1 obtuvimos un tiempo promedio de 82.238,67 ms; con 0,5 disminuyó a 12.697,28 ms; con 0,7 obtuvimos 13.451,95 ms y con 0,9 se redujo a 7.852,33 ms.

Aunque los resultados no disminuyeron de manera perfectamente continua, sí observamos una tendencia general: utilizar un factor de carga muy bajo, especialmente 0,1, aumentó considerablemente el tiempo necesario para cargar el catálogo. Al permitir una mayor ocupación de la tabla, los tiempos fueron mucho menores.

En separate chaining el cambio fue menos pronunciado. Obtuvimos 7.638,22 ms con factor 2,0; 6.428,06 ms con 4,0; 6.614,96 ms con 6,0 y 6.712,89 ms con 8,0. En este caso, los resultados se mantuvieron relativamente cercanos entre sí, por lo que concluimos que el cambio del factor de carga tuvo un impacto menor sobre el tiempo que en linear probing.

6. ¿Qué cambios percibe en el consumo de memoria al modificar el factor de carga máximo?

En este aspecto encontramos una tendencia mucho más clara: al aumentar el factor de carga máximo, disminuyó el consumo de memoria en ambos mecanismos.

En linear probing obtuvimos 2.607.968,61 kB con factor 0,5, 1.968.559,07 kB con 0,7 y 1.616.115,13 kB con 0,9. La medición correspondiente a 0,1 no pudo completarse, por lo que no la utilizamos para establecer esta comparación.

En separate chaining observamos el mismo comportamiento: pasamos de 1.105.585,51 kB con factor 2,0 a 738.400,08 kB con 4,0, 615.629,80 kB con 6,0 y finalmente 557.264,82 kB con 8,0.

A partir de esto entendemos que, al permitir que la tabla alcance una mayor ocupación antes de redimensionarse, se necesita reservar menos espacio para manejar la misma cantidad de información.

7. ¿Qué cambios percibe en el tiempo de ejecución al modificar el esquema de colisiones? Si los percibe, describa las diferencias y argumente su respuesta.

Percibimos diferencias importantes. Separate chaining mostró tiempos más estables que linear probing durante nuestras pruebas.

Los tiempos de separate chaining se mantuvieron entre 6.428,06 ms y 7.638,22 ms, a pesar de cambiar el factor de carga. En linear probing encontramos una variación mucho mayor, principalmente por el resultado obtenido con el factor 0,1, que alcanzó 82.238,67 ms. Incluso al excluir este caso, los factores 0,5 y 0,7 presentaron tiempos superiores a los registrados con separate chaining, mientras que con 0,9 linear probing se acercó más a sus resultados.

Por lo tanto, a partir de nuestras mediciones consideramos que separate chaining tuvo un comportamiento más estable respecto al tiempo de ejecución, mientras que linear probing fue mucho más sensible al factor de carga utilizado.

8. ¿Qué cambios percibe en el consumo de memoria al modificar el esquema de colisiones? Si los percibe, describa las diferencias y argumente su respuesta.

En todas las mediciones que logramos completar, separate chaining presentó un consumo menor que linear probing.

En linear probing, incluso el menor valor obtenido fue de 1.616.115,13 kB con factor 0,9. En separate chaining, el mayor consumo registrado fue 1.105.585,51 kB con factor 2,0 y llegó a disminuir hasta 557.264,82 kB con factor 8,0.

Esto nos permitió observar que, para nuestra implementación y los archivos large utilizados en el laboratorio, separate chaining aprovechó la memoria de una manera más favorable. Consideramos que esto se relaciona con la forma en que cada mecanismo maneja las colisiones: linear probing necesita mantener posiciones disponibles directamente dentro de la tabla para realizar el sondeo, mientras que separate chaining permite almacenar varios elementos asociados a una misma posición.